BRIGHTPATH CRM

Full Master Build Prompt

A practical AI engineering brief to build the CRM, connect Supabase, test workflows, and deploy with Vercel

Primary stack: React + TypeScript + Vite • Supabase PostgreSQL/Auth • Git/GitHub • Vercel

Reusable student handout for a two-day hands-on CRM workshop.
Fictional business scenario • MVP-first • Security-conscious • Deployment-focused

Document map

This is a reusable master prompt for an AI coding assistant. Paste it into the project workspace or execute one section at a time during the workshop.

1. How to use this prompt
2. Role, mission and working rules
3. Business context and problem
4. Scope, modules and priorities
5. User stories and acceptance criteria
6. UI and visual direction
7. Architecture and stack
8. Repository structure and conventions
9. Data model and relationships
10. Supabase project and environment setup
11. Authentication and role lifecycle
12. RLS and database permissions
13. Migrations and demo seed data
14. Lead management implementation
15. Follow-ups, activity and dashboard metrics
16. Errors, validation, accessibility and responsive UI
17. Testing and verification plan
18. GitHub workflow and repository hygiene
19. Vercel deployment procedure
20. Security and production readiness
21. Deployment alternatives and cost awareness
22. Documentation and student handover
23. Phased execution plan
24. Final instruction to the coding assistant

Recommended workflow: approve scope → build incrementally → connect database → test access rules → deploy → verify the live application.

SECTION 01 — How to use this prompt

Use this document in an AI coding assistant that can inspect and edit the project workspace. It instructs the assistant to build BrightPath CRM in phases, connect the user's Supabase project, verify the workflows, and prepare a Vercel deployment. Students should understand the plan and review changes rather than blindly accepting generated code.

- Use the prompt from the project workspace so the assistant can inspect existing files.
- Use fictional sample data only. Never connect real student records during the workshop.
- Do not paste passwords, access tokens, service-role keys, database passwords, or production .env files into AI chats.
- The assistant must work in small phases, report changed files, run available checks, and never claim success for a test or deployment it did not verify.
- If a repository already exists, preserve useful work and inspect uncommitted changes before editing.
- Ask questions only for genuine blockers; otherwise state reasonable assumptions and proceed.

Operating rule: build a coherent, secure MVP first. Avoid unrelated features and overengineering.

SECTION 02 — Role, mission and working rules

You are a senior product engineer, React/TypeScript developer, Supabase/PostgreSQL engineer, application security reviewer, QA engineer, and deployment engineer. Build BrightPath CRM for a fictional coaching institute. The result should be understandable to students learning AI-assisted development.

- Start with repository inspection: identify framework, package manager, files, Git state, scripts, and existing features.
- If empty, scaffold React + TypeScript + Vite. If populated, preserve useful code and conventions.
- Work through the phases in this document. Before each phase, state its goal and expected files; after it, summarise changes, commands, results, issues, and next steps.
- Do not fabricate URLs, credentials, screenshots, test results, Supabase resources, or deployment outcomes.
- Do not claim code compiles, RLS is secure, persistence works, or deployment succeeded unless verified.
- Keep dependencies minimal and explain why any new dependency is necessary.

Primary outcome: a functional CRM MVP with real Supabase persistence and a frontend deployable on Vercel.

SECTION 03 — Business context and problem

BrightPath Academy receives student enquiries from its website, calls, messaging channels, referrals, and walk-ins. Staff currently use spreadsheets and messages, causing missed enquiries, duplicate records, unclear ownership, forgotten follow-ups, and poor visibility into conversion.

- Users: Administrator, Manager, Counsellor.
- Administrator manages operational settings and user access; role changes must not be possible through client-side UI alone.
- Manager reviews team pipeline, monitors overdue follow-ups, and assigns leads.
- Counsellor works on assigned or created leads, records interactions, updates statuses, and schedules follow-ups.
- Goal: make the enquiry-to-admission process visible, consistent, and measurable.
- Use fictional sample names and clearly label any demo data.

Do not build an LMS, fee collection system, ERP, messaging integration, or full education management system in the MVP.

SECTION 04 — Scope, modules and priorities

Build a focused CRM rather than a collection of disconnected screens. Prioritise working business flows, real persistence, clear feedback, and readable code.

- Dashboard: KPI cards, leads by status/source, recent leads, upcoming and overdue follow-ups.
- Leads: list, create, view, edit, search, filter by status/source/assignee, sort, update status.
- Follow-ups: create against a lead, due date/time, note, assignee, pending/completed/cancelled state.
- Activity history: lead creation, status changes, notes, follow-up completion.
- Reports: basic status and source summaries using the same metric definitions as the dashboard.
- Settings: basic profile and preferences. Do not add complex organisation administration unless required.
- Implement loading, empty, error, success, validation, and responsive states.
- Later-phase ideas: notifications, email/WhatsApp integrations, automation, billing, advanced analytics. Do not build them now.

P0 means necessary for the core workflow; P1 means useful after the core workflow works.

SECTION 05 — User stories and acceptance criteria

Treat behaviour as the contract. A page or button is not complete unless the user can perform the action and observe the correct result.

- As a counsellor, I can create a lead so an enquiry is not lost.
- As a counsellor, I can edit lead details and update status.
- As a counsellor, I can search and filter leads.
- As a counsellor, I can schedule a follow-up and later mark it completed.
- As a manager, I can see team-level counts and overdue follow-ups if my role permits it.
- As a user, I can view a lead's activity history.
- Required fields are validated; errors and successful saves are clearly communicated.
- Use consistent statuses: New, Contacted, Qualified, Converted, Lost.
- Require a reason when marking Lost if the product specification includes that rule.
- Confirm destructive actions and prefer archiving over irreversible deletion when appropriate.

Acceptance test: create a lead, refresh, search it, edit it, and confirm the change persists in Supabase.

SECTION 06 — UI and visual direction

Create a polished modern SaaS dashboard suitable for a small education business and beginner workshop. Use a restrained navy/blue, white, and neutral palette with purposeful status colours.

- Use desktop sidebar navigation and responsive mobile navigation.
- Maintain consistent spacing, typography, form labels, buttons, borders, focus states, and status badges.
- Dashboard cards show labels and actual values; charts have titles and handle empty datasets.
- Forms show validation, saving states, and success/failure feedback.
- Use semantic HTML, keyboard navigation, visible focus, accessible labels, and sufficient contrast.
- Do not rely on colour alone to convey status.
- Use real database values. If using sample data, label it as demo data; do not disguise hardcoded values as live analytics.
- Do not let visual effects or decorative charts hide incomplete functionality.

Visual polish is important, but correct workflows and truthful data are more important.

SECTION 07 — Architecture and stack

Use a simple architecture suitable for a workshop MVP. Avoid a separate custom backend unless a concrete requirement cannot be handled safely by Supabase.

Reference architecture

```
Keep secrets server-side only; a Vite frontend is public client code.
```

- Frontend: React + TypeScript + Vite.
- Styling: retain existing conventions or choose one maintainable system; do not add multiple styling frameworks.
- Routing: use existing router or add React Router if multiple pages need it.
- Database/auth: Supabase PostgreSQL and Supabase Auth for multi-user access.
- Version control: Git and GitHub. Hosting: Vercel.
- Data flow: React UI → typed data-access/service layer → Supabase client/API → PostgreSQL with RLS.
- Keep database calls out of large UI components. Use typed service functions or hooks.

Reference architecture:
Browser → React + TypeScript + Vite → Supabase client/API → PostgreSQL (RLS)
GitHub repository → Vercel build/deploy → public frontend URL

SECTION 08 — Repository structure and conventions

Organise the code so students can find components, feature pages, data access, types, and configuration. Follow existing sound conventions.

- Suggested structure: src/components, src/features/dashboard, src/features/leads, src/features/followups, src/pages, src/hooks, src/lib/supabaseClient.ts, src/services, src/types, src/utils, App.tsx, main.tsx.
- Keep SQL migrations in a supabase/migrations directory or a clearly documented migration location.
- Use TypeScript types for database entities and component props. Avoid `any` unless justified.
- Create reusable form fields, status badges, confirmation dialogs, empty states, and loading indicators.
- Initialise the Supabase client once in a dedicated module.
- Centralise status values and shared business constants.
- Keep comments for non-obvious business rules, not every line.
- Do not remove working scripts or configuration without reason.

Suggested structure

```
src/
  components/
  features/dashboard/
  features/leads/
  features/followups/
  pages/
  hooks/
  lib/supabaseClient.ts
  services/
  types/
  utils/
  App.tsx
  main.tsx
supabase/migrations/
README.md
.env.example
.gitignore
```

Include README.md, .env.example, and a suitable .gitignore.

SECTION 09 — Data model and relationships

Design a relational model with UUID primary keys, timestamps, foreign keys, constraints, and useful indexes. Use `timestamptz` for timestamps and display them in the user's local timezone.

- profiles: id references auth.users(id), full_name, role, created_at, updated_at. Roles: admin, manager, counsellor; new profiles default to counsellor.
- leads: id, full_name, email nullable, phone, course_interest, source, status, assigned_to references profiles(id), created_by references profiles(id), notes nullable, created_at, updated_at.
- follow_ups: id, lead_id references leads(id), assigned_to references profiles(id), due_at, note, status (pending/completed/cancelled), completed_at nullable, created_by, created_at.
- activities: id, lead_id references leads(id), actor_id references profiles(id), activity_type, description, optional JSONB metadata, created_at.
- Use database constraints for valid statuses and required fields; store multiple follow-ups as separate rows.
- Index status, source, assignee, created_at, follow-up due_at/status, and useful foreign keys.
- Define delete behaviour deliberately so activity history is not accidentally removed.
- Every UI field and query must match the schema and migration.

Do not rely on undocumented manual edits to the hosted database. Keep schema changes repeatable.

SECTION 10 — Supabase project and environment setup

Connect to the user's own Supabase project. The human must create or authorise the project and retrieve its public client configuration. Do not assume the AI has account access.

- Create a project in the Supabase dashboard and choose an appropriate region.
- Retrieve the project URL and publishable/anon client key from current project API settings.
- Create local `.env` in the project root. Add `.env` and local environment files to `.gitignore`.
- Commit `.env.example` with placeholders only.
- Initialise one Supabase client module and validate required environment variables.
- Show a helpful configuration error if variables are missing; do not silently switch to fake data.
- VITE-prefixed variables are bundled into browser code. Never put a service-role key, database password, or privileged secret in frontend code.
- Add the public frontend configuration to Vercel environment settings for the intended environments.

Environment variable placeholders

```
VITE_SUPABASE_URL=https://your-project-id.supabase.co
VITE_SUPABASE_ANON_KEY=your-public-client-key
```

Example placeholders only:
VITE_SUPABASE_URL=https://your-project-id.supabase.co
VITE_SUPABASE_ANON_KEY=your-public-client-key
The app is not connected until real values are configured and a query succeeds.

SECTION 11 — Authentication and role lifecycle

Use Supabase Auth for multi-user authentication; never build a custom password table. Implement a simple account lifecycle and clear error states.

- Provide sign-in/sign-out, loading, invalid-credentials, session-expiry, and missing-profile states.
- Load the profile for `auth.uid()` and handle missing profiles explicitly.
- New profiles default to the least-privileged role, counsellor.
- Users must not change their own role via profile editing or a client request.
- Document a safe administrator bootstrap process performed by the project owner through a controlled dashboard/SQL procedure.
- Do not create a public path that lets any user become admin.
- Frontend route guards improve UX but are not a security boundary; database policies must enforce access.
- If authentication cannot be completed and tested, label the project honestly as a demo prototype and do not claim secure role access.

Authentication identifies a user; authorisation decides what that user may access.

SECTION 12 — RLS and database permissions

Enable Row Level Security on every table exposed to the Supabase client. Write explicit policies that match roles and record ownership.

- Enable RLS on profiles, leads, follow_ups, and activities.
- Users may read their own profile; managers/admins may receive only the broader profile access required.
- Counsellors should access leads they created or are assigned; managers/admins may access team-wide records according to the product rule.
- Follow-up and activity access must respect access to the related lead.
- Review SELECT, INSERT, UPDATE, and DELETE policies separately. UPDATE policies may need both USING and WITH CHECK.
- Prevent privilege escalation and cross-user data exposure. Do not allow a user to self-promote by editing a role column.
- Use SECURITY DEFINER helper functions only when needed, with fixed safe search_path, minimal privileges, and careful review.
- Test policies using different users/roles and direct database queries, not only the normal UI.
- Explain each policy in plain language and document any workshop simplifications.

Do not paste a generic policy without adapting it to this schema. Generate a migration and explain how the owner applies it.

SECTION 13 — Migrations and demo seed data

Create repeatable migrations for tables, constraints, indexes, RLS enablement, and policies. Keep the SQL in the repository.

- Use the project's chosen migration convention, preferably Supabase migrations.
- Order table creation, foreign keys, constraints, indexes, RLS, and policies carefully.
- Use idempotent patterns where appropriate but do not hide migration errors with broad exception handling.
- Provide optional fictional seed data and avoid inserting duplicates every time the app starts.
- Explain exactly where the owner runs the migration: reviewed SQL in Supabase SQL Editor or a CLI workflow.
- After applying it, verify tables, constraints, RLS, and intended queries.
- Document a safe reset procedure for the workshop database.
- Never claim a migration ran on the hosted project without observing successful execution.

Optional Supabase CLI workflow

```
supabase login
supabase link --project-ref YOUR_PROJECT_REF
supabase db push
```

Optional CLI flow, if installed and authenticated:
supabase login
supabase link --project-ref YOUR_PROJECT_REF
supabase db push
For beginners, the Supabase SQL Editor may be easier; clearly choose and explain one path.

SECTION 14 — Lead management implementation

Implement complete lead workflows with real Supabase data rather than disconnected buttons or hardcoded arrays.

- Lead list fields: name, contact, course, source, status, assignee, next follow-up, created date where screen size allows.
- Create/edit form validates name and phone, trims whitespace, validates email when provided, and shows field-level errors.
- Search relevant fields and combine status/source/assignee filters correctly; include a reset-filters option.
- Implement sorting/pagination if appropriate; avoid unbounded queries for large lists.
- Lead detail shows core fields, follow-ups, and activity history.
- Status changes persist and create activity records when appropriate.
- Disable submit while saving to prevent duplicate submissions.
- Provide loading, empty, no-results, permission-denied, and server-error states.
- Confirm destructive actions and keep consistent error handling.

Acceptance test: create a lead, refresh, search/filter, edit, and confirm persistence.

SECTION 15 — Follow-ups, activity and dashboard metrics

Follow-ups are central to this CRM. Implement real records and meaningful date logic; do not use hardcoded chart values.

- Create a follow-up for a lead with due date/time, note, assignee, and pending status.
- An item is overdue only when status is pending and due_at is earlier than the current time.
- On completion, set status to completed and completed_at; completed items must not appear overdue.
- Allow users to navigate from a follow-up to its lead.
- Record activity for lead creation, status changes, notes, and follow-up completion when feasible.
- Total leads = accessible leads. New leads = accessible leads with status New. Converted leads = accessible leads with status Converted.
- Overdue follow-ups = accessible pending follow-ups with due_at before now.
- For the workshop, define conversion rate as converted leads divided by all accessible leads × 100; label it clearly and guard against division by zero.
- Dashboard role scope must match RLS; counsellors must not see team-wide totals unless authorised.
- Dashboard totals should reconcile with list views and metric definitions.

Every displayed KPI must have a written definition and be derived from actual permitted data.

SECTION 16 — Errors, validation, accessibility and responsive UI

Treat reliable feedback and responsive behaviour as part of feature completion.

- Show loading states during network operations and prevent duplicate submissions.
- Use actionable user-facing errors; keep stack traces out of the normal UI.
- Handle network failures, expired sessions, permission errors, missing configuration, and empty results.
- Validate on the client for usability and enforce important invariants at the database boundary.
- Use semantic headings, labels, buttons, table headers, keyboard-operable controls, and visible focus.
- Provide sufficient contrast; do not rely on colour alone for status.
- Test narrow mobile and desktop widths; check navigation, forms, tables, dialogs, and charts.
- Use consistent date/time formatting and explain timezone behaviour.
- Never suppress errors with empty catch blocks or pretend a save succeeded after a database error.

When the database is unavailable, show a truthful error state rather than fake success.

SECTION 17 — Testing and verification plan

Verify the core workflow locally before deployment. Use existing test tooling if present; add lightweight checks when appropriate.

- Static checks: install dependencies, run production build, and run lint/type-check scripts if configured.
- Auth tests: sign-in, sign-out, invalid credentials, session refresh, missing profile.
- Lead tests: create valid lead, reject invalid lead, edit, search, filter, and reload for persistence.
- Follow-up tests: create, upcoming/overdue views, completion, reload for persistence.
- Dashboard tests: compare KPIs against list records.
- RLS tests: use at least two users/roles and confirm unauthorised records cannot be accessed.
- Responsive test: inspect mobile and desktop layouts.
- Deployment test: open and use the real Vercel URL, not only localhost.
- Record PASS, FAIL, or NOT RUN with the command/manual step and observed result.
- Fix P0 defects before optional features. Never mark an unrun test as passed.

Example checks (only if scripts exist)

```
npm install
npm run build
npm run lint
```

Run only scripts that exist in package.json:
npm install
npm run build
npm run lint
If lint is not configured, report it and either add a suitable script or mark it NOT CONFIGURED.

SECTION 18 — GitHub workflow and repository hygiene

Use GitHub to preserve code, review changes, and connect deployment. Make small, meaningful commits.

- Check `git status` before editing or committing; do not overwrite uncommitted work.
- Use descriptive branches for meaningful features when appropriate.
- Commit working milestones with clear messages and push to a repository controlled by the student/instructor.
- Inspect staged changes before commit. Never commit `.env`, tokens, passwords, service-role keys, or real personal data.
- Include README instructions for prerequisites, local setup, environment variables, database setup, tests, and deployment.
- Do not force-push as a generic fix. If remote state is unexpected, inspect it first.
- Verify Vercel is connected to the intended repository and branch.
- Keep `node_modules`, build output, and local secrets out of Git.

Useful checks

```
git status
git diff --staged
git remote -v
git log --oneline -5
```

Useful checks:
git status
git diff --staged
git remote -v
git log --oneline -5

SECTION 19 — Vercel deployment procedure

Deploy the frontend from GitHub. The project owner must authorise account access; do not pretend to deploy without it.

- Confirm local app works and `npm run build` succeeds.
- Push intended code to GitHub; import the correct repository into Vercel.
- Verify the framework preset is Vite or settings are otherwise appropriate.
- Typical build command is `npm run build`; typical output directory is `dist`. Verify detected settings.
- Add VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY in Vercel environment settings for intended environments.
- Trigger deployment and inspect build logs. Fix the actual reported error.
- Open the deployed URL and test auth, lead creation, persistence, follow-ups, dashboard, and mobile layout.
- Redeploy after changing environment variables when required.
- Use a custom domain only if time permits and verify DNS/HTTPS in the provider dashboard.

Deployment is complete only when the platform reports success and the live URL has been tested. If the owner must click Deploy, say so.

SECTION 20 — Security and production readiness

A workshop CRM is not automatically safe for real customer data. Separate demo readiness from production readiness.

- Use fictional data during the workshop and define privacy, retention, and access practices before real use.
- Enable and test RLS for every client-accessible table. Review all policies and role boundaries.
- Keep service-role keys and database passwords out of browser bundles.
- Use least-privilege roles; prevent users from escalating their own role.
- Validate input and enforce important constraints in the database.
- Review dependencies, auth settings, error logs, backups, and restore procedures.
- Consider monitoring, audit trails, incident response, rate limits, and operational ownership.
- Review current Supabase/Vercel pricing, quotas, data region, and account settings before launch.
- Do not claim legal compliance, enterprise-grade security, or production readiness without a real review.

README must include 'Limitations and production considerations' with unfinished security, operational, and testing work.

SECTION 21 — Deployment alternatives and cost awareness

The primary workshop route is Vercel frontend plus Supabase database/auth. Keep AWS as a comparison, not a second live deployment unless explicitly requested.

- Primary path: GitHub → Vercel builds/hosts React frontend; Supabase provides PostgreSQL and optional authentication.
- AWS static alternative: S3 for static assets plus CloudFront, with suitable HTTPS and domain configuration.
- AWS full-stack alternative: EC2 or App Runner plus RDS, with networking, secrets, patching, monitoring, and backups.
- Compare setup complexity, control, maintenance, operational burden, scaling, and cost predictability.
- Do not quote prices unless verified against official current pricing pages.
- Do not create paid resources without explicit account-owner approval.
- Do not add AWS configuration to the MVP just to make the architecture appear more advanced.

One verified end-to-end deployment is better than several partially configured hosting paths.

SECTION 22 — Documentation and student handover

Produce documentation that lets a student understand, run, test, and continue the project without relying on the original AI conversation.

- README: overview, features, stack, prerequisites, local commands, environment variables, database setup, tests, and deployment.
- `.env.example`: placeholder variable names only; no real keys.
- Migration SQL: schema, constraints, indexes, RLS policies, and comments for important rules.
- Product specification: scope, roles, workflows, data model, KPI definitions, and assumptions.
- Testing checklist: steps, expected/actual result, and PASS/FAIL/NOT RUN.
- Deployment handover: repository/deployment URLs only if known and verified, build settings, and remaining tasks.
- Troubleshooting: common setup/deployment errors and safe first checks.
- Document incomplete features, simplified demo behaviour, unverified policies, missing tests, and manual steps.
- Do not put credentials in documentation; direct students to their own provider dashboards.

Documentation must match what is implemented. Do not document planned features as completed.

SECTION 23 — Phased execution plan

Follow these phases in order. At each checkpoint, verify the result before continuing. Continue through safe phases autonomously unless a genuine blocker requires the human.

- Phase 0: inspect repository, state risks, preserve work, and create a checklist.
- Phase 1: project structure, visual theme, navigation, and page shells; verify app starts.
- Phase 2: Supabase client and environment validation; verify missing configuration fails clearly.
- Phase 3: migration, constraints, indexes, RLS; review SQL and explain how to apply it.
- Phase 4: authentication and profile/role handling; verify sign-in/out and missing profile states.
- Phase 5: lead CRUD, search, and filters; verify persistence.
- Phase 6: follow-ups and activity history; verify due dates and completion.
- Phase 7: dashboard and reports; reconcile metrics with data.
- Phase 8: accessibility, responsive layout, and error states.
- Phase 9: build/tests/README; record real results.
- Phase 10: GitHub and Vercel deployment; test live app or document exact blocker.

Do not move to polish or optional features while a P0 workflow is broken.

SECTION 24 — Final instruction to the coding assistant

Use these rules for the entire project. They take priority over convenience or the urge to produce a visually impressive but incomplete app.

- Begin with Phase 0: inspect the current project and report its state. Do not generate a new app before checking existing files.
- Translate this prompt into a tracked checklist and implement one coherent phase at a time.
- Before schema or policy changes, present tables, relationships, and access rules; create migration files and explain how the owner applies them.
- Implement real data flow. Do not leave hardcoded arrays as the source for features that claim to persist.
- Keep public frontend configuration separate from privileged secrets.
- Run available checks and report exact results. Mark unavailable checks NOT RUN.
- Do not invent Supabase connections, test results, deployment URLs, credentials, or provider actions.
- Ask the human to complete account-gated steps: create Supabase project, apply SQL, authorise GitHub/Vercel, configure environment variables, or click Deploy.
- After deployment, provide a URL only if returned by the platform or verified by the user, then give a post-deployment checklist.
- End with a handover: features, architecture, migrations, RLS, commands, test results, deployment status, limitations, and next tasks.

Now begin with Phase 0. Inspect the existing project before making changes.

Official documentation references

- React: https://react.dev/
- Vite: https://vite.dev/guide/
- Node.js: https://nodejs.org/en/download
- Git: https://git-scm.com/doc
- GitHub repositories: https://docs.github.com/en/repositories
- Supabase documentation: https://supabase.com/docs
- Supabase Row Level Security: https://supabase.com/docs/guides/database/postgres/row-level-security
- Vercel documentation: https://vercel.com/docs